

OpenBox Auditoria Tecnica, Estudo de Caso e Plano de Execucao

Documento AV01-A

Baseado no workspace local /home/aops/OpenBox0.1v

ABSTRACT

Este documento reorganiza o projeto OpenBox segundo os criterios do enunciado AV01-A: estudo de caso, justificativa tecnica, demandas de infraestrutura, evidencias de desenvolvimento e plano de execucao. A analise parte do material ja existente no projeto e adiciona uma auditoria objetiva do estado atual do prototipo. Conclusao central: a proposta e tecnicamente valida como projeto de infraestrutura residencial focado em privacidade e monitoramento, mas o repositorio ainda exige correcoes de bootstrap, rede e publicacao de evidencias publicas antes de ser tratado como stack pronta para implantacao.

1. Objetivo do documento

O OpenBox e uma plataforma open source de home theater para Raspberry Pi 4 com VPN, DNS criptografado, bloqueio de trackers, monitoramento local e operacao auditavel. O objetivo deste PDF e atender ao formato da AV01-A com um artefato unico que apresente:

- o problema real que pode ser tratado por infraestrutura de TI;
- a proposta tecnica do OpenBox;
- as demandas de hardware e software;
- uma auditoria honesta do estado atual do projeto;
- um plano de orquestracao para evolucao e entrega.

2. Estudo de caso: problema e justificativa

O estudo de caso parte de um problema comum no ambiente domestico em 2026: consumo de streaming concentrado em dispositivos e ecossistemas fechados, normalmente acompanhados de telemetria, ACR em smart TVs, dependencia de servicos centralizados e pouca transparencia operacional. Ao mesmo tempo, o custo acumulado de multiplas assinaturas e a ausencia de uma pilha auditavel abrem espaco para uma solucao de infraestrutura local.

O OpenBox responde a tres lacunas tecnicas.

1. Privacidade de rede. O usuario residencial geralmente nao controla resolucao DNS, telemetria de firmware ou vazamento de trafego quando a VPN falha.
2. Observabilidade. Plataformas fechadas nao expoem estado de servicos, consumo, temperatura, disponibilidade nem trilhas claras de falha.
3. Reprodutibilidade. Projetos divulgados como "DIY" costumam parar em anotacoes soltas, instalacao manual e claims tecnicos nao verificaveis.

Nesse contexto, o OpenBox se justifica como um projeto de infraestrutura de TI porque combina servicos de rede, sistema operacional, automacao, seguranca e monitoramento em um unico dispositivo de borda.

3. Solucao proposta

A proposta do OpenBox e substituir a funcao "caixa-preta" da TVBox comercial por uma pilha aberta, modular e documentada. O dispositivo usa Raspberry Pi 4 como host de servicos e separa o sistema em quatro camadas: midia, privacidade de egress, tuning de kernel e auditoria/monitoramento.

```
+-----+
| CAMADA 4 - AUDITORIA E MONITORAMENTO           |
| Netdata | Uptime Kuma | Monit | Lynis | RKHunter | ntfy  |
+-----+
| CAMADA 3 - OTIMIZACAO DO HOST                   |
| BBR | CAKE | IRQ affinity | MTU 1420 | sysctl hardening |
+-----+
| CAMADA 2 - PRIVACIDADE DE REDE                   |
| Pi-hole -> dnscrypt-proxy -> WireGuard           |
| Tor restrito a metadados e lookup, nao ao stream |
+-----+
| CAMADA 1 - MIDIA                                 |
| Stremio em container, exposto somente na LAN via Caddy |
+-----+
```

Essa arquitetura atende ao criterio do enunciado porque descreve um problema concreto, delimita contexto operacional e justifica tecnicamente o uso de camadas de rede, software e observabilidade.

4. Demandas de infraestrutura

Categoria	Item	Papel no projeto
Hardware	Raspberry Pi 4 4GB	Host principal da pilha
Hardware	SSD USB 3.0 64GB+	Sistema e persistencia
Hardware	Fonte 5.1V/3A	Estabilidade eletrica do host
Hardware	Case com ventilacao ativa	Controle termico
Rede	Conexao banda larga	Transporte para VPN e streaming
Rede	LAN domestica	Acesso aos paineis e ao player
Software	Raspberry Pi OS Lite ARM64	Base do sistema
Software	WireGuard	Tunel VPN com kill switch
Software	Pi-hole + dnscrypt-proxy	Filtragem e criptografia DNS
Software	Tor	Metadados e lookup por SOCKS5
Software	Docker + Stremio	Servico principal de midia
Software	Caddy	Proxy reverso e consolidacao de acesso
Software	Netdata, Uptime Kuma, Monit	Observabilidade e watchdogs
Software	Fail2ban, AIDE, Lynis	Hardening e auditoria local

O OpenBox tambem exige configuracoes complementares que o documento original do projeto ja identifica corretamente: MTU de 1420 para o tunel, CAKE para controle de bufferbloat, BBR para tuning de throughput e bind local para paineis administrativos.

5. Auditoria do estado atual do projeto

A auditoria do workspace mostra um projeto bem documentado, com boa divisao por fases de instalacao, threat model explicito e um vault de Obsidian integrado para gestao do conhecimento. Em termos academicos, isso e um ponto forte: existe justificativa, arquitetura, backlog, runbook e artefatos de suporte.

Ao mesmo tempo, o prototipo atual ainda nao pode ser tratado como implantacao concluida. Os principais achados sao:

1. Bootstrap SSH inseguro para primeira instalacao.
O script 'install.d/00-base.sh' aplica o hardening de SSH e reinicia o servico, mas 'etc/ssh/sshd_config.d/99-openbox.conf' restringe acesso a 'AllowUsers openbox' sem que esse usuario seja criado no repositório. Em hardware real, isso pode bloquear o acesso remoto imediatamente apos a instalacao.
2. Configuracao Tor ainda depende de placeholder invalido.
'etc/tor/torrc.example' traz 'HashedControlPassword' como placeholder e 'install.d/05-tor.sh' reinicia o Tor antes da substituicao por um hash valido. O efeito pratico e uma fase de instalacao nao deterministica.
3. Proxy administrativo ainda nao e implantavel sem ajuste manual.
'etc/caddy/Caddyfile' depende de 'REPLACE_WITH_BCRYPT_HASH' para o usuario administrativo e o script de monitoramento aplica esse arquivo imediatamente. Sem esse passo manual, o acesso consolidado aos paineis nao fica pronto.
4. Roteamento policy-based do WireGuard esta incompleto.
O arquivo 'etc/wireguard/wg0.conf.example' cria a tabela 51820 e a regra 'not fwmark', mas a configuracao ainda nao fecha todos os detalhes necessarios para um full tunnel robusto e gerenciavel. A arquitetura esta correta, mas a operacionalizacao ainda precisa endurecimento.
5. O modo --dry-run ainda nao e totalmente confiavel.
Alguns scripts escrevem em arquivos do sistema fora da funcao 'run()', como em 'install.d/02-nftables.sh' e 'install.d/06-stremio.sh'. Para um projeto que se vende como auditavel, o modo de simulacao precisa ser realmente nao destrutivo.
6. Evidencias publicas ainda nao foram publicadas.
O diretorio atual nao esta inicializado como repositório Git e nao possui remote GitHub configurado. Portanto, a parte do enunciado que pede links de desenvolvimento ainda nao esta plenamente atendida no formato exigido para submissao.

Resultado da auditoria: o OpenBox e forte como proposta tecnica e como estudo de caso de infraestrutura, mas ainda esta em fase de consolidacao operacional.

6. Pontos fortes identificados

- Escopo bem definido: streaming sob VPN; Tor limitado a metadados e lookup.
- Separacao clara entre docs, scripts de instalacao, configuracoes e validacoes.
- Threat model e trade-offs declarados de forma honesta.
- Obsidian vault integrado ao projeto, o que melhora rastreabilidade e gestao.
- Plano de hardening ja alinhado com criterios de aceite e validacao tecnica.

7. Plano de orquestracao e desenvolvimento

Para transformar o OpenBox de um prototipo forte em um entregavel tecnico maduro, o caminho recomendado e sequencial. Abaixo esta o plano executivo:

Fase	Objetivo	Entregavel
Fase 1	Corrigir bootstrap e simulacao	SSH sem lockout e dry-run nao mutante
Fase 2	Fechar rede e servicos base	WireGuard, Tor e Caddy com configuracao valida
Fase 3	Endurecer plano de monitoramento	Fail2ban, watchdogs e validacoes coerentes
Fase 4	Publicar evidencias	Repositorio Git, URL publica e anexos do projeto
Fase 5	Fechar entrega academica	Pitch de 5 min, PDF final e demonstracao guiada

Em ordem pratica, a prioridade nao deve ser "adicionar mais funcionalidades", mas estabilizar o caminho de instalacao e a prova de funcionamento. Em especial:

1. criar ou parametrizar o usuario administrativo antes de aplicar 'AllowUsers';

2. substituir placeholders secretos por processo de bootstrap seguro;
3. validar o tunel WireGuard com testes reais de egress e kill switch;
4. publicar o projeto em Git com evidencias HTTP/HTTPS antes da submissao.

8. Evidencias de desenvolvimento disponiveis

Os artefatos tecnicos presentes no workspace e uteis para a entrega sao:

```
README.md
docs/CASE_STUDY.md
docs/HARDENING_PLAN_v2.md
docs/THREAT_MODEL.md
docs/REFERENCES.md
docs/OBSIDIAN_VAULT.md
vault/Dashboards/OpenBox Vault Home.md
vault/Delivery/OpenBox Delivery Backlog.md
vault/Delivery/OpenBox Roadmap.md
tools/sync_obsidian_vault.py
tests/validate-obsidian-vault.sh
```

Limite atual da evidencia: o workspace local ainda nao contem um repositorio Git publicado. Isso significa que o requisito "links de desenvolvimento" esta atendido apenas internamente por caminhos locais e documentacao local. Antes da submissao final, e necessario publicar o projeto em uma URL publica e substituir esta observacao por links HTTP/HTTPS verificaveis.

9. Aderencia aos criterios do enunciado AV01-A

- Qualidade tecnica: atendida pelo estudo de caso, arquitetura, threat model e plano de hardening.
- Uso de ferramentas e evidencias: parcialmente atendido com docs, scripts, vault e validacoes locais; pendente para GitHub publico.
- Estrutura e escrita tecnica: atendida por este PDF com problema, contexto, arquitetura, auditoria, plano e evidencias.

10. Conclusao

O OpenBox e um bom projeto para AV01-A porque resolve um problema real com uma solucao tecnicamente defensavel e bem documentada. A banca nao deve receber esse material como "produto final pronto", mas como uma plataforma em desenvolvimento com arquitetura correta, lacunas conhecidas e plano claro de consolidacao.

Se a prioridade for maximizar a nota da entrega, a melhor estrategia e simples: manter o discurso tecnico honesto, corrigir os bloqueios de bootstrap, publicar as evidencias em GitHub e usar este PDF como documento principal da fase de documentacao e prototipagem.

11. Referencias tecnicas utilizadas

- WireGuard whitepaper: <https://www.wireguard.com/papers/wireguard.pdf>
- Pi-hole com dnscrypt-proxy: <https://docs.pi-hole.net/guides/dns/dnscrypt-proxy/>
- Netdata behind Caddy: <https://learn.netdata.cloud/docs/netdata-agent/configuration/securing-agents/running-the-agent-behind-a-reverse-proxy/caddy>
- Raspberry Pi 4 documentation: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- Bibliografia completa do projeto: 'docs/REFERENCES.md'

Data do documento: 2026-04-18.